TinyRVV: A Minimalist's Vector Subset for RISC-V

Jerry Zizhuo Yun
The University of British Columbia
Vancouver, Canada
zizhuoyun@gmail.com

Guy Lemieux
The University of British Columbia
Vancouver, Canada
lemieux@ece.ubc.ca

Abstract

TinyRVV is a minimalist vector-processing extension for the CORE-V CV32E20, also known as CVE2, a compact 32-bit RISC-V processor. The design implements a minimal RISC-V Vector (RVV) instruction subset that includes vector configuration, unit-stride vector load and store, and vector add/multiply/and/shift-right-logical, along with some fixed RVV parameters (SEW=32, LMUL=1, and VLEN=256). The underlying microarchitecture is not SIMD; vector instructions execute one 32-bit element at a time while sharing the primary ALU of the CVE2 in order to save area. Performance of the implementation is evaluated using cycle-accurate Verilator simulation on three benchmarks — packed rgba2luma, planar rgb2luma, and matmul8. Area and clock speed are evaluated using Vivado synthesis targeting an AMD UltraScale+ FPGA. Despite the lack of SIMD parallelism, TinyRVV still shows a performance gain up to 4.2× due to a reduction in loop overhead and redundant loads and stores of data. Overall, this project demonstrates that a tiny application-specific subset of RVV can be implemented on a small embedded processor with area increase limited to 10% area and a 10% reduction in clock speed but an improvement in clock cycle counts between 1.9× and 4.2×. While TinyRVV provides the basic vector instructions and infrastructure needed, specific applications may wish to implement a handful of additional instructions to meet their needs.[1]

CCS Concepts

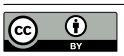
• **Computer systems organization → Single instruction, multiple data**.

Keywords

TinyRVV, RISC-V, RISC-V Vectors, RVV-Lite, vector extension, low-area design, hardware reuse, FPGA implementation, application-specific processor

ACM Reference Format:
Jerry Zizhuo Yun and Guy Lemieux. 2026. TinyRVV: A Minimalist's Vector Subset for RISC-V. In *Proceedings of the 16th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies (HEART 2026), June 17–19, 2026, Heidelberg, Germany*. ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/3814576.3814588

[1]TinyRVV source code can be found at https://github.com/UBC-ORCA/cve2-tinyrvv



HEART 2026, Heidelberg, Germany

ACM ISBN 979-8-4007-2452-7/26/06
https://doi.org/10.1145/3814576.3814588

1 Introduction

Many embedded and image-processing workloads perform repetitious computation across vectors of data. On a compact scalar processor, these kernels are executed as repeated sequences of loads, arithmetic operations, and stores for each data element, plus incrementing of loop index variables and pointers, and branches. This scalar instruction overhead limits efficiency. Instead, vector processing provides a natural way to accelerate repetitive computations, where performance can be improved by reducing the number of explicit loads, stores, pointer increments, and loop operations and making these implicit instead. Loads and stores can often be replaced by reusing data held in vector registers, which are much larger than scalar registers. Meanwhile, loop operations are replaced with vector instruction control logic, and pointer increments are replaced with hardware-based address generation units [1, 3, 5]. However, full-featured vector architectures often introduce significant hardware complexity and area cost, making them less attractive for small, embedded, in-order cores designed to fit within tight implementation constraints.

This trade-off is especially relevant for the CORE-V CV32E20 [9], also known as CVE2, a lightweight two-stage pipelined 32-bit RISC-V processor that targets resource-constrained systems. Rather than pursuing a complete RISC-V Vector (RVV) implementation, this work explores whether a minimal RVV subset can deliver useful acceleration while preserving the simplicity of the base core.

With over 400 instructions, and many modes, RVV is large and complex. Some variants of RVV, such as Zve32x, are made smaller by omitting all floating-point and all 64b integer operations, but it still includes multiple data element sizes (8/16/32), saturating and widening arithmetic, reductions, indexed/strided/segmented loads and stores, and multiply-add instructions. Even RVV-Lite [8], a proposal that defines layered subsets of RVV, minimally requires over 80 instructions. Many of these are unnecessary for simple embedded applications.

In this paper, we design a minimal vector extension for CVE2 called TinyRVV. The design supports vector configuration (set vector length), unit-stride vector loads and stores, basic integer operations (add, addi, and), and shift-right logical. It is designed to share the ALU and multiplier with its scalar host, so it operates only on 32b elements without any register groups (in RVV, this means SEW=32 and LMUL=1). These design choices favour a low area overhead implementation in a small scalar core.

In TinyRVV, the length of each vector register is 256 bits (VLEN=256) and the implementation executes one 32b element per cycle using the (unmodified) shared ALU/multiplier. Additional logic implements hardware to iterate over the vector length and a separate vector register file. While CVE2 uses flip-flops for its register file, the FPGA-optimized TinyRVV implementation uses BRAM instead.

The vector register file is small enough (8192 bits) to not overburden even the smallest FPGA BRAMs (typically 4096 bits).

Across three benchmark kernels, the TinyRVV design achieves speedups from 1.9× to 4.2× while adding approximately 10% in LUT count relative to the scalar baseline. The results suggest that even a lightweight vector extension can provide meaningful acceleration with reasonable area overhead in a compact RISC-V processor.

2 Related Work

This work was inspired by RVV-Lite [8], a proposal that breaks down RVV into disjoint sub-extensions consisting of 7 primary layers, named A1 through A7, plus 5 optional sub-extensions, plus 3 extra layers. Sub-extensions A1 and A4 are further subdivided into 7 and 4 groups, respectively, denoted by a lower-case letter, e.g. (a) through (g). These subextensions and subdivisions are organized primarily on a structural basis, so instructions that share the same logic are grouped together, but there is some functional organization as well, e.g. A.1.(a) includes only enough instructions to set the vector length plus do vector loads and stores, forming a minimal set of instructions that can copy data in and out of the vector register file but perform no computation. However, RVV-Lite has not been proven to align with the end-user needs for different applications or application domains, for example.

Arrow [2] implements a subset of the RVV 0.9 specification, but it is not driven by native CPU instructions; it connects to a closed-source MicroBlaze (non-RISC-V) via memory-mapped I/O. It includes unit-stride and strided memory access, single-width integer[2] addition, subtraction, multiplication, and division; bitwise logic and shift; and integer compare, min/max, merge, and move operations. With an execution width of 64 bits, it operates at 112MHz on a Xilinx Artix 7 FPGA (28nm, -1 speed grade), and requires just 474 LUTs. The resources used to implement its vector register file, multiplier, or divider are not presented.

Stream Semantic Registers [11] (SSR) add new semantics to RISC-V scalar registers – it implicitly recodes register reads and writes as strided memory accesses, avoiding the need for a vector register file and instead backing the vector in main memory. For an 11% area overhead in an ASIC implementation, speedups of 3–5× are reported. Together with a zero-overhead loop instruction, SSR achieves many of the same objectives as TinyRVV with the exception that SSR relies upon the cache for performance, whereas TinyRVV uses the standard RVV ISA with a vector register file.

Lightweight Vector Extensions [6] (LVE), implemented in TinBiNN [7] on an FPGA together with a CNN accelerator, also resembles TinyRVV. Instead of a vector register file with named vector registers, it uses an addressable scratchpad with pointers, like VectorBlox MXP [12]. It is also not RVV compatible.

Compared to more complete RISC-V vector processors such as Ara [3], Vicuna [10], Saturn [13] and Gatling-V [4], TinyRVV targets a much smaller footprint with a minimal vector ISA.

3 TinyRVV Design and Implementation

TinyRVV implements a minimal subset of vector instructions for configuration, computation, and memory accesses. To do this, it extends CVE2 with a minimalist vector unit targeted at regular integer computation. The goal is not to implement a full RVV processor, but to identify a minimal (micro-)architectural starting point that still captures useful speedup on selected workloads. The TinyRVV design therefore implements only these specific RVV instructions: `vsetvl`, `vsetvli`, `vsetivli`, `vle32.v` and `vse32.v` (unit stride load and store), `vadd.vv|vx`, `vmul.vx`, `vand.vx|vi`, and `vsrl.vi` (shift-right-logical).[3] These instructions are a minimal set needed to execute the three matrix-oriented benchmarks in this paper.

To simplify the vector control logic, and to reuse the scalar ALU without modification, TinyRVV only operates on 32-bit elements. The vector control unit expands each vector instruction as a sequence of 32-bit elements, up to the vector length, and sends them to the execution stage. It also controls an address generation unit used by vector load and store instructions. Logic in the scalar execution stage is reused for vector add, multiply and shift instructions. Adding certain vector instructions, such as `vxor`, or operand sources, such as `vand.vv`, impose minimal overhead, but they were omitted in this design to limit the TinyRVV ISA as much as possible.

Each vector register in the implementation is 256 bits long. The 32 named registers total 8192 bits and within most FPGAs this fits with 1 BRAM in 2R1W mode; some tiny FPGAs with fewer bits and limited read/write port configurations will need up to 4 BRAMs. Note that CVE2's register file is implemented using discrete flip-flops; we did not attempt to optimize the scalar part of the CVE2 implementation for FPGAs. For ASIC implementations where BRAM is not available, limiting TinyRVV to 8 or 16 vector registers can save considerable area.

The baseline CVE2 implementation has two scalar pipeline stages. At the top processor level, the original cve2_core, cve2_decoder, and cve2_id_stage were modified to issue vector instructions to the vector unit, and retire them only after the vector unit signals completion. The vector unit therefore behaves as a multi-cycle execution side path attached to the scalar core. To improve timing, an extra pipeline stage is inserted between the vector control/operand-selection logic and the shared scalar execution path. This breaks a long combinational path into the ALU/multiplier while preserving one-element-per-cycle steady-state execution for single-cycle vector ALU operations. Branches use CVE2's normal flush mechanism: if a branch redirects the pipeline, wrong-path vector instructions are squashed before they are issued, so they cannot update the vector register file or other architectural state. Other than the additional pipeline stage, no intensive efforts were made to optimize the implementation for area or speed.

4 Results

The original CVE2 and a modified version with TinyRVV were implemented with Vivado ML2025.2 targetting an AMD/Xilinx ZCU104 UltraScale+ XCZU7EV-2FFVC1156 FPGA (16nm, -2 speed grade). Area is reported from out-of-context flattened synthesis to allow optimizations across some hierarchical boundaries. Performance is evaluated with cycle-accurate Verilator simulation using computation-only cycle counts collected from start and end marker instructions placed around each benchmark kernel.

[2] Arrow appears to implement only eight 8 bit operations packed into 64 bit words.

[3] Operand sources are `vv` (vector-vector), `vx` (vector-scalar) or `vi` (vector-immediate).

Table 1: Results using AMD ZCU104 UltraScale+ (16nm).

Design	LUTs	FFs	BRAM (URAM)	DSPs	F_{max}
Arrow [2]	474	773	0 (0)	–	112 MHz†
RVV-Lite [8], A1(a)	1220	n/a	20 (2)	0	177 MHz
RVV-Lite [8], A1(a)(b)	2584	n/a	36 (2)	0	181 MHz
RVV-Lite [8], A1 to A4(c)	5165	n/a	37 (2)	8	167 MHz
TinyRVV, no ALU	345	212	1 (0)	0	197 MHz
CVE2, no vector ops	3650	2343	0 (0)	1	188 MHz
CVE2 + TinyRVV	3995	2555	1 (0)	1	169 MHz

† Xilinx Artix 7 FPGA (28nm)

Table 2: Cycle-count speedup, TinyRVV versus scalar CVE2.

packed rgba2luma	planar rgb2luma	matmul8
1.88×	2.01×	4.16×

4.1 Area Results

Table 1 summarizes the main area results. The RVV-Lite implementation [8] targets the same FPGA. The minimal A1(a) sub-division, which only supports vector configuration, load and store, uses 1220 LUTs, while the configuration needed to run the three benchmarks in this paper, all layers to A4(c), uses 5165 LUTs. In contrast, TinyRVV uses just 345 LUTs to execute these benchmarks. While the RVV-Lite approach includes more instructions, it costs much more to implement than the minimal subset. Application-specific processors should consider adding only the required instructions, and no more. For comparison, the size of CVE2 before and after adding TinyRVV is also shown. CVE2 itself is not FPGA-optimized.

4.2 Performance Results

Three benchmarks are used: packed rgba2luma, planar rgb2luma, and matmul8. The packed rgba2luma benchmark reads one 32 bit input word containing three 8 bit colour channels (ignoring the fourth byte), and computes an 8 bit luminance value stored in a 32 bit output word. In contrast, the planar rgb2luma benchmark reads three 32 bit input words from three separate matrices, each containing four 8 bit colour values in the 32 bit word. The matmul8 benchmark multiplies two 8×8 matrices of 32 bit integers; the vectorized version loads data only once when computing $\mathbf{C} = \mathbf{A} \cdot \mathbf{B}$ as an outer product (in $i \cdot k \cdot j$ loop order) — all rows of $\mathbf{B}$ are preloaded into vector registers, while a row-vector of $\mathbf{C}$ is loaded, accumulated into using the product between rows of $\mathbf{B}$ and individual elements from a row of $\mathbf{A}$, and stored. These kernels are typical embedded computations with differences in data size, arithmetic intensity, and data reuse. The scalar baselines are compiled C programs built with $-O2$, while the vector kernels are hand-written assembly code.

Table 2 summarizes benchmark speedups. The best result is matmul8, where the entire input matrix $\mathbf{B}$ can be held in vector registers across many multiply-add iterations. The two rgb-to-luma benchmarks show that data layout matters: packed rgba2luma performs per-element unpacking from one load, while planar rgb2luma requires loading from three separate input arrays.

5 Conclusion

This paper presented TinyRVV, a minimalist RISC-V vector subset for small embedded processors. Implemented in CVE2, it re-uses the ALU but adds a vector register file and all required vector control logic to execute vector operations at one 32-bit word per cycles. The maximum vector length of 8 words, implying a vector register file of 8192 bits (VLEN=256), is sufficient for small matrices. Compared with baseline CVE2, it adds roughly 10% LUT overhead while delivering approximately 1.9× to 4.2× computation speedup across the benchmark set, suggesting that a minimal vector extension may be a practical enhancement for small embedded RISC-V processors targetting specific workloads. To cover other applications or application domains, more instructions can be added; in particular, instructions that re-use the ALU and re-use existing operand sources require very little additional logic. Future work will explore adding these and other extra instructions while keeping area minimal. Applications that need more instructions should evaluate the cost of implementation versus the overhead of using existing instructions.

Acknowledgments

This work was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC) Grant RGPIN-2022-05377. Additional support was provided by Andes Technology Canada Corporation.

References

[1] K. Asanović. 1998. *Vector Microprocessors.* Ph. D. Dissertation. University of California, Berkeley.

[2] I. Assir, M. Iskandarani, H. Sandid, and M. Saghir. 2021. Arrow: A RISC-V Vector Accelerator for Machine Learning Inference. *CoRR* abs/2107.07169 (2021). arXiv:2107.07169 https://arxiv.org/abs/2107.07169

[3] M. Cavalcante, F. Schuiki, F. Zaruba, M. Schaffner, and L. Benini. 2020. Ara: A 1-GHz+ Scalable and Energy-Efficient RISC-V Vector Processor With Multiprecision Floating-Point Support in 22-nm FD-SOI. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* (2020).

[4] F. Chalabi and G. Lemieux. 2026. Gatling-V: An FPGA-based RISC-V Vector Core with Single-Issue, Multiple In-Flight Instruction Execution. In *ACM International Symposium on FPGAs.*

[5] D. Dabbelt, B. Keller, and K. Asanović. 2016. Vector Processors for Energy-Efficient Embedded Systems. In *Proceedings of the Third ACM International Workshop on Many-Core Embedded Systems.*

[6] G. Lemieux. 2016. ORCA-LVE: Embedded RISC-V with Lightweight Vector Extensions. In *Proceedings of the 4th RISC-V Workshop.*

[7] G. Lemieux, J. Edwards, J. Vandergriendt, A. Severance, R. De Iaco, A. Raouf, H. Osman, T. Watzka, and S. Singh. 2019. TinBiNN: Tiny Binarized Neural Network Overlay in about 5,000 4-LUTs and 5mW. arXiv:1903.06630 [cs.DC] https://arxiv.org/abs/1903.06630

[8] G. Lemieux, C. White, F. Alves, and F. Chalibi. 2023. Poster: RVV-Lite v0.5: A Modest Proposal for Reducing the RISC-V Vector Extension. https://riscv-europe.org/summit/2023/posters. In *RISC-V Summit Europe* (Barcelona).

[9] OpenHW Group. [n. d.]. CORE-V CVE2 RISC-V IP. https://github.com/openhwgroup/cve2. Last accessed 2026/04/22.

[10] M. Platzer and P. Puschner. 2021. Vicuna: A Timing-Predictable RISC-V Vector Coprocessor for Scalable Parallel Computation. In *33rd Euromicro Conference on Real-Time Systems (ECRTS 2021)*, Vol. 196. doi:10.4230/LIPIcs.ECRTS.2021.1

[11] F. Schuiki, F. Zaruba, T. Hoefler, and L. Benini. 2019. Stream Semantic Registers: A Lightweight RISC-V ISA Extension Achieving Full Compute Utilization in Single-Issue Cores. *CoRR* abs/1911.08356 (2019). arXiv:1911.08356 http://arxiv.org/abs/1911.08356

[12] A. Severance and G. Lemieux. 2013. Embedded supercomputing in FPGAs with the VectorBlox MXP matrix processor. In *IEEE/ACM/IFIP International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS '13).* Article 6, 10 pages.

[13] J. Zhao, D. Grubb, M. Rusch, T. Wei, K. Anderson, B. Nikolic, and K. Asanovic. 2024. Instruction Scheduling in the Saturn Vector Unit. arXiv:2412.00997 [cs.AR] https://arxiv.org/abs/2412.00997